



Ben-Gurion University of the Negev
Faculty of Computer and Information Science

Methods for Detecting Attack

Final Project

Submitted by

Natalie Mirelashvili
Faculty of Computer and Information Science
Ben-Gurion University of the Negev
E-mail: miralshv@post.bgu.ac.il

[complete others]

June 9, 2026

1 Related Work

1.1 Prompt injection and automated attack generation

Prompt injection attacks have become a major security concern for LLM-integrated systems because trusted instructions and untrusted content are often processed in the same model context [1, 2]. Indirect prompt injection showed that malicious instructions can be embedded in external content, such as web pages or documents, and later activated when an LLM application consumes that content [1]. PromptRobust [3] further showed that small perturbations at the character, word, sentence or semantic level can significantly affect model behavior. These results show that attack success depends on the model, task, prompt structure and location of attacker-controlled input.

Manual prompt injections often require substantial effort and can be brittle across models and application settings [4, 5] which motivated automated attack generation. Universal adversarial suffixes showed that optimization-based search can generate token sequences that transfer across aligned models, including black-box systems [4].

JudgeDeceiver extended this idea to LLM-as-a-Judge systems by searching for adversarial sequences that cause the judge to prefer a target response [5]. These studies show that effective attacks can be generated automatically rather than manually crafted, making malicious behavior increasingly difficult to identify.

Recent works extends automated attack generation from static optimization to reinforcement learning, which is especially relevant for multi-step and agentic settings. AutoInject [6] formulates prompt injection as an RL problem and learns universal, transferable suffixes while jointly optimizing attack success and benign-task utility. This shows that attackers can learn reusable injection strategies from reward signals instead of hand-engineering prompts. Slingshot [7] applies a related RL approach to agent-to-agent jailbreaking, where success is measured by verifiable tool execution rather than subjective textual compliance. Together, these works frame reinforcement learning as a general mechanism for automating adversarial discovery when attackers can exploit structured rewards and behavioral feedback.

1.2 Defenses for LLM-integrated systems

Existing defenses for LLM-integrated systems are commonly divided into soft and hard defenses [2, 8]. This taxonomy captures the distinction between mitigation techniques applied at inference time and defenses that modify the underlying system design.

Soft defenses. Soft defenses aim to mitigate prompt injection without modifying the underlying model or application architecture [2]. Liu et al. [2] benchmark a diverse set of soft defenses, ranging from prompt modifications to inference-time detection techniques.

For indirect prompt injection, BIPIA [9] introduces boundary-awareness mechanisms and explicit reminders that encourage the model to treat retrieved content as data rather than executable instructions. These methods are practical and easy to deploy, but their effectiveness often decreases against adaptive attacks or when models fail to consistently follow prompt-level safeguards [2, 9].

Hard defenses. In contrast, hard defenses seek to address prompt injection at the architectural level by enforcing stronger trust boundaries, context separation, or input authenticity guarantees [10, 11]. The StruQ [11] framework separates system instructions and user data into structured channels and fine-tunes the model to follow only the instruction channel [10]. By construction, this approach treats prompt injection as a context-separation problem rather than solely a malicious-text detection problem. Shield adopts a broader system perspective by defining integrity, source identification, attack detectability and utility preservation as required safety properties for LLM-integrated applications. Survey studies place these mechanisms within a wider defense toolbox that also includes corpus cleaning, robustness optimization and safe instruction tuning [8].

Overall, recent work suggests a shift from prompt-level mitigation toward architectural defenses that provide stronger guarantees through context separation, authenticated data flows and safer-by-design interfaces [9, 10, 11].

1.3 Threat modeling and automation for LLM systems

As LLM-integrated applications introduce new attack surfaces, several works have adapted classical threat-modeling methodologies to capture risks that are unique to AI systems. Tete et al. [12] propose a threat-modeling framework that combines STRIDE, DREAD, and Shostack’s four-question framework to analyze threats such as prompt injection, data poisoning, jailbreaking and information disclosure. Building on this direction, Gadade et al. [13] argue that the dynamic and adaptive nature of LLM systems requires automated threat-modeling capabilities. Their framework integrates machine learning, NLP, behavioral profiling and anomaly detection to automatically identify, assess and prioritize security risks in LLM deployments.

STRIDE GPT [14] explores the automation of the threat-modeling process by generating threat models, attack trees, mitigations and security test cases from system descriptions or source code context. By reducing the expertise required to perform threat analysis, such systems aim to make security assessment more accessible and scalable.

Xu et al. [15] demonstrated that formal threat models can be transformed into attack paths and executable security tests, enabling systematic validation of security requirements. Similarly, Tuma et al. [16] showed that security design flaws can be detected automatically by applying model-query patterns to annotated Data Flow Diagrams. The Meta Attack Language further extends this direction by providing domain-specific lan-

guages for reusable threat modeling and attack simulation [17]. Collectively, these works illustrate a broader shift from manually constructed threat models toward automated frameworks that can generate, analyze, and operationalize security knowledge throughout the system lifecycle.

References

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection,” 2023.
- [2] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX Security Symposium (USENIX Security 24)*, USENIX Association, 2024.
- [3] K. Zhu, J. Wang, J. Zhou, Z. Wang, H. Chen, Y. Wang, L. Yang, W. Ye, Y. Zhang, N. Z. Gong, and X. Xie, “Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts,” 2023.
- [4] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” 2023.
- [5] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, “Optimization-based prompt injection attack to llm-as-a-judge,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS ’24*, (New York, NY, USA), pp. 660–674, Association for Computing Machinery, 2024.
- [6] X. Chen, J. Zhang, and F. Tramèr, “Learning to inject: Automated prompt injection via reinforcement learning,” 2026.
- [7] S. Nellessen and T. Kachman, “David vs. goliath: Verifiable agent-to-agent jailbreaking via reinforcement learning,” 2026.
- [8] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, “A survey on large language model (llm) security and privacy: The good, the bad, and the ugly,” *High-Confidence Computing*, vol. 4, no. 2, p. 100211, 2024.
- [9] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, “Benchmarking and defending against indirect prompt injection attacks on large language models,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD ’25*, Association for Computing Machinery, 2025.

- [10] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “Struq: Defending against prompt injection with structured queries,” 2024. To appear at USENIX Security Symposium 2025.
- [11] F. Jiang, Z. Xu, L. Niu, B. Wang, J. Jia, B. Li, and R. Poovendran, “Identifying and mitigating vulnerabilities in llm-integrated applications,” 2023.
- [12] S. B. Tete, “Threat modelling and risk analysis for large language model (llm)-powered applications,” 2024.
- [13] B. Gadade and G. A. Patil, “A threat modeling framework for llm system integration leveraging nlp and machine learning,” *Communications on Applied Nonlinear Analysis*, vol. 32, no. 10s, 2025.
- [14] M. Adams and P. Raghavan, “Se radio 648: Matthew adams on ai threat modeling and stride gpt.” Software Engineering Radio podcast, 2024.
- [15] D. Xu, M. Tu, M. Sanford, L. Thomas, D. Woodraska, and W. Xu, “Automated security test generation with formal threat models,” *IEEE Transactions on Dependable and Secure Computing*, vol. 9, no. 4, pp. 526–540, 2012.
- [16] K. Tuma, L. Sion, R. Scandariato, and K. Yskout, “Automating the early detection of security design flaws,” in *Proceedings of the ACM/IEEE 23rd International Conference on Model Driven Engineering Languages and Systems, MODELS ’20*, pp. 332–342, Association for Computing Machinery, 2020.
- [17] P. Johnson, R. Lagerström, and M. Ekstedt, “A meta language for threat modeling and attack simulations,” in *Proceedings of the 13th International Conference on Availability, Reliability and Security, ARES 2018*, pp. 38:1–38:8, Association for Computing Machinery, 2018.